

Fussgängerstreifen-Klassifikation auf SwissImage 10 cm

Projektbericht, FHGR Deep-Learning Hackathon FS 2026

Mike Obrietan

Juni 2026

a) Eckdaten Datensatz

Die Aufgabe ist eine binäre Klassifikation. Für jede 25 m × 25 m grosse Kachel aus SwissImage 10 cm (bei 10 cm/px sind das nativ etwa 250×250 px) soll das Modell entscheiden, ob ein Fussgängerstreifen vorhanden ist (1) oder nicht (0).

Den Datensatz haben wir im Kurs gemeinsam erstellt. Jede und jeder hat mit dem SwissImage Annotator Tool einen geografischen Ausschnitt annotiert (Ascona-Locarno, Basel, Bern, Freiburg, Genf, Zürich), danach wurden die Teile zusammengeführt. Insgesamt sind es 112'137 Kacheln mit 3'159 Positives und 108'978 Negatives, also ein Klassenverhältnis von etwa 1:34.5. Diese Zusammenarbeit ist laut Aufgabenstellung erlaubt; ich weise sie offen aus, sie gibt eine grössere und geografisch breitere Basis als ein Einzeldatensatz.

Klassenverteilung pro Split (geografisch getrennt, mehr dazu in Abschnitt c):

Split	Kacheln	Positives	Pos-Rate
Train	76'839	1'943	2.53 %
Val	15'546	573	3.69 %
Test	15'741	511	3.25 %
Puffer gedroppt	4'011	n/a	n/a

Dass die Positiv-Rate zwischen den Splits schwankt (2.5 bis 3.7 %), kommt vom räumlichen Split; bei zufälligem Split wäre sie gleich, aber die Test-Werte zu optimistisch.

Zur Datenqualität habe ich ein paar Checks gemacht (`src/data/stats.py`, Ausgaben in `reports/data/`):

- Bildgrössen: Die Stichprobe von 500 Bildern (`size_check.json`) läuft auf dem Manifest mit den gecachten Pfaden und zeigt durchgehend 224×224. Die nativen Rohkacheln sind vor dem Cache etwa 250×250 (10 cm/px × 25 m). Keine Ausreisser.
- Duplikate (perceptual hash): nur 2 Gruppen mit 4 Bildern, alle uniforme Strassen- oder Wiesenkacheln in den Negatives. Vernachlässigbar.
- Annotations-Risiko: Weil mehrere Personen annotiert haben, ist „Streifen sichtbar“ nicht immer gleich definiert (verblasste Streifen, randständig, Privatgrund). Die grosse geografische Streuung mildert das etwas ab, weil kein Annotator einen Split dominiert (siehe auch Fehleranalyse in d).
- Eine technische Stolperfalle: In den Drive-ZIPs lagen macOS-Dateien (`.*.png`), an denen PIL abstürzt. Nach einem Filter in `cache.py` und `ingest.py` blieben genau 3'159 / 108'978 übrig.

Wichtig ist vor allem die Imbalance, nur etwa 3 % Positives. Ein Trivialmodell, das immer „kein Streifen“ sagt, kommt schon auf 96.75 % Accuracy bei einer PR-AUC von 0.032. Accuracy als Hauptmetrik wäre also irreführend.

b) Architektur des Modells

Ich habe drei Modelle in PyTorch gebaut (`src/models/`), alle mit 224×224 RGB als Eingabe.

1. Baseline CNN (etwa 1.17 Mio Parameter): 4 Conv-Blöcke mit 32, 64, 128, 256 Kanälen (je $2 \times [3 \times 3 \text{ Conv} + \text{BN} + \text{ReLU}]$) und danach 2×2 MaxPool, dann Global Average Pooling, Dropout(0.3) und Linear(256, 1). Das ist meine ehrliche Vergleichsbasis ohne Pretraining.
2. ResNet18 als Transfer-Modell (etwa 11.2 Mio Parameter, He et al. 2015, *Deep Residual Learning*) mit ImageNet-Gewichten (Russakovsky et al. 2015). Den FC-Layer habe ich durch Dropout(0.3) + Linear(512, 1) ersetzt. Das Fine-Tuning läuft in zwei Stufen: in Epoche 1 bis 3 ist das Backbone eingefroren und nur der Head wird trainiert (lr 1e-3), ab Epoche 4 taue ich das Backbone auf und trainiere mit unterschiedlichen Lernraten weiter (Backbone 1e-4, Head 1e-3). Das ist Standardpraxis und schützt die ImageNet-Features vor den grossen Gradienten eines anfangs zufälligen Heads.
3. ResNet18 mit Focal Loss als Ablation.

Warum diese Wahl: Die Skip-Connections im ResNet lösen das Vanishing-Gradient-Problem tiefer Netze (He et al. 2015), und das ImageNet-Pretraining bringt allgemeine Low- und Mid-Level-Filter mit, die auch auf Luftbildern funktionieren. ResNet18 ist gross genug für 224×224, aber klein genug, dass es bei nur 3'159 Positives nicht sofort überfittet. Domänennäher wäre Remote-Sensing-Pretraining (z. B. SatMAE, Cong et al. 2022, oder torchgeo, Stewart et al. 2022); im Zeitrahmen nicht umgesetzt, siehe Ausblick in e).

c) Optimierungen

Ich habe die Methoden bewusst zur Aufgabe und zu den Daten gewählt und nicht einfach Defaults übernommen.

Geo-Split statt zufällig (`src/data/split.py`): Luftbilder sind räumlich autokorreliert, weil benachbarte Kacheln oft denselben Strassenzug und dieselbe Beleuchtung teilen. Ich lege ein 500-m-Gitter über die LV95-Koordinaten und droppe an den Split-Grenzen alle Kacheln im 25-m-Puffer, das sind 4'011 Stück. Daraus ergibt sich Train/Val/Test = 76'839 / 15'546 / 15'741.

Augmentation (nur im Training, `src/data/dataset.py`): RandomRotation(180°) mit fill=128, dazu horizontaler und vertikaler Flip und leichtes ColorJitter. Das ist datengetrieben, weil Luftbilder keine feste Ausrichtung haben, also darf ich voll rotieren.

fill=128 statt schwarz verhindert, dass das Modell die schwarzen Ecken nach einer Rotation als Negativ-Merkmal lernt. Crops und Shears habe ich weggelassen, weil sie die dünnen Linien-Labels zerstören, MixUp ebenfalls, weil es die Labels ungültig macht.

Imbalance, ein Mechanismus pro Lauf: Primär nutze ich einen WeightedRandomSampler mit pos_ratio=0.3 zusammen mit BCEWithLogitsLoss. Damit sind etwa 30 % Positives pro Batch drin, also rund 12-faches Oversampling. Als Ablation nehme ich Focal Loss (Lin et al. 2017, $\gamma=2$, $\alpha=0.25$) auf der natürlichen Verteilung, das gewichtet schwere Beispiele stärker, statt zu oversampeln.

Threshold-Tuning (`src/evaluate.py`): Ich suche die F1-optimale Schwelle auf dem Val-Set, speichere sie fix und werte damit einmal auf dem Test-Set aus. Der Default 0.5 ist bei dieser Imbalance fast immer schlecht (Baseline landet bei 0.93, Focal bei 0.45).

Restliches Setup: AdamW (lr 1e-3, weight decay 1e-4), Cosine-Schedule, Mixed Precision (etwa 1.6× schneller), Early Stopping auf der Val-PR-AUC (Patience 5), Seed 42. Dazu ein Pre-Resize-Cache (`cache.py`), der die Bilder einmalig von 250 auf 224 bringt. Damit steigt die GPU-Auslastung von etwa 50 % auf 95 bis 100 % (mit nvidia-smi geprüft).

d) Resultate

Modell	Imbalance	Val PR-AUC	Test PR-AUC	Test F1	Recall	Precision	Acc.
Majority („immer 0")	n/a	n/a	0.033	0.000	0.000	0.000	0.968
Baseline CNN	Sampler+BCE	0.969	0.965	0.923	0.953	0.895	0.995
ResNet18 Transfer	Sampler+BCE	0.966	0.966	0.915	0.953	0.881	0.994
ResNet18 Transfer	Focal Loss	0.977	0.976	0.917	0.945	0.890	0.994

Die Test-Confusion des besten Modells: TP=483, FN=28, FP=60, TN=15'170. Es findet also 483 von 511 Positives (95 %). Alle PR- und ROC-Kurven, Confusion-Plots, Trainingsverläufe und die Top-Fehlerbilder liegen in `reports/runs/<run>/eval/`.

Was ich daraus lese: (i) Alle Modelle schlagen die Majority-Baseline um etwa Faktor 30 in der PR-AUC. (ii) Die Test-PR-AUC liegt höchstens 0.005 unter der Val-PR-AUC. Der Geo-Split (Puffer = 1 Kachel, 25 m) reduziert räumliche Leakage also stark; ganz ausschliessen kann ich sie nicht, weil Rest-Leakage entlang derselben Strasse über die Puffergrenze hinaus möglich bleibt. Die direkte Kontrolle Geo- gegen Random-Split steht noch aus. (iii) Focal ist bei der PR-AUC etwas besser als Sampler+BCE (+0.011), die Wahl der Imbalance-Strategie macht also einen messbaren Unterschied. (iv) Beim F1 ist überraschend das Baseline-CNN vorne. Zebra-Streifen sind ein einfaches, lokales Muster (periodische helle Balken auf Asphalt), und da bringt mehr Tiefe wenig.

Fehleranalyse: Ich habe pro Klasse die 20 nach Konfidenz schlimmsten Fehlerbilder des besten Modells angeschaut (`reports/runs/resnet18_focal/eval/errors/`) und grob einsortiert, die Kategorien überschneiden sich teilweise.

False Positives (20): etwa 8× Fahrbahn oder Kreuzung mit Strassenmarkierungen (Haltelinien, Spurmarkierungen, Pfeile), etwa 6× Dächer oder Gebäudestrukturen mit periodischem Muster (Wellblech- oder Solardächer, Treppen), etwa 4× Parkplätze oder markierte Flächen, etwa 2× Vegetation oder Schatten.

False Negatives (20): etwa 11× Streifen, die nur einen kleinen oder randständigen Bildanteil ausmachen, während Wasser, Dach, Bäume oder ein Parkplatz die Kachel dominieren, etwa 4× Streifen in starkem Schatten, etwa 3× zwischen Fahrzeugen verdeckt, etwa 2× neben rot/orangen Velostreifen.

Kritisch eingeordnet: Was die Fehlerkategorien nahelegen, ist, dass das Modell auf das periodische Hell-Dunkel-Muster reagiert, laut den False Positives aber auch auf allgemeine Fahrbahnmarkierungen und periodische Dächer anspringt, also nicht nur auf Zebrastreifen. Das ist eine Interpretation aus den Fehlerbildern; eine echte Saliency- oder Grad-CAM-Analyse, die zeigt, worauf das Modell tatsächlich schaut, habe ich nicht gemacht und steht als nächster Schritt aus. Der häufigste Grund für False Negatives ist nicht Verblassung, sondern dass der Streifen bei einer 25-m-Kachel mit einem Label fürs ganze Bild oft nur ein winziger Randbereich ist. Das ist eher ein Label- und Auflösungsthema als ein reines Modellproblem. Die Zahlen stammen aus einem einzelnen Run und können sich mit Mehr-Seed-Läufen leicht verschieben.

e) Persönliche Erkenntnisse

Was mich überrascht hat: Mein erster Reflex war „ResNet18 schlägt die Baseline klar". Das stimmte nicht, der Vorteil ist klein und nur in der PR-AUC sichtbar. Mehr Parameter helfen also nicht automatisch, und es lohnt sich, vorher zu fragen, ob die Aufgabe das überhaupt braucht. Sampler und Focal klangen für mich zuerst gleichwertig, sind sie aber nicht. Focal kalibriert die Wahrscheinlichkeiten besser (Schwelle 0.45 statt 0.79). Und Threshold-Tuning ist nicht nur Kosmetik, mit dem Default 0.5 würde ich rund 4 F1-Punkte liegen lassen.

Herausforderungen und wie ich sie gelöst habe: (1) Beim Datenformat habe ich zuerst die echte Struktur angeschaut (zwei Ordner mit LV95-Koordinaten im Dateinamen), statt ein COCO- oder VOC-Format anzunehmen. Das hat mir eine Neu-Implementierung erspart. (2) Der erste Lauf hing bei 50 % GPU-Auslastung (DataLoader durch PNG-Dekodieren/Resizen limitiert); das Pre-Cache-Skript (`cache.py`) brachte sie auf 95–100 % und halbierte die Epochenzzeit. (3) Die macOS-Dateien in den ZIPs liessen PIL abstürzen, gelöst mit einem `.*`-Filter.

Was ich nächstes Mal anders machen würde: (i) Remote-Sensing-Pretraining (torchgeo statt ImageNet) für domänennähere Features. (ii) Test-Time Augmentation (Inferenz auf 4 Rotationen und Scores mitteln) als günstigen Recall-Boost. (iii) Hard-Negative-Mining auf die häufigsten False Positives (Parkplätze, Solardächer) in einer zweiten Runde. (iv) Cross-City-Generalisierung testen (auf 3 Städten trainieren, auf den anderen 3 testen) — härter und näher am echten Einsatz.

Mein Fazit: Bei stark imbalancierten Aufgaben ist die Pipeline (Geo-Split, Imbalance-Strategie, Threshold-Tuning, ehrliche Metrik) wichtiger als die Architektur. Eine schlampige Pipeline mit ResNet50 schlägt eine saubere Pipeline mit einem kleinen CNN nicht.